

The Final Three: Completing the NSL Compiler-Native Architecture

Papers 11-13: Fault Tolerance (CPFT), Quantization Synthesis (CTQS), and Architecture Generalization (CNAG)

Authors: Brandon Wiemer

Date: April 2026

Status: Research Proposal

Paper 11: Compile-Planned Fault Tolerance (CPFT)

How the Static Binary Survives a Hardware Fire

1. The Problem

CPDT and WGGO produce a mathematically optimal, static communication schedule across a cluster. But at 1,024 H100 GPUs, Meta reports MTBF of ~3 hours. ByteDance's ByteRobust identifies ~38,000 failures in 3 months on 9,600 GPUs. If node 847 drops mid-backward, a perfectly static compiled binary crashes — the reduce-scatter expects 1024 participants and gets 1023.

NSL already has the runtime infrastructure for fault tolerance (M58): UDP heartbeat monitoring with Alive/Suspect/Dead/Excluded states, three-tier hierarchical checkpointing (GPU→CPU→NVMe→remote), and elastic DP resize (adjusting `grad_accum_steps` to preserve effective batch size). What's missing is the **compiler integration** — how WGGO's static plan interacts with runtime elasticity.

2. Three Compiler-Native Innovations

2.1 WGGO-Optimal Checkpoint Injection

The problem: Where in the training loop should checkpoints occur? Too frequent wastes bandwidth. Too infrequent risks losing hours of progress. ByteRobust achieves <0.9% overhead with every-step checkpointing, but this requires careful scheduling.

CPFT's approach: WGGO treats checkpoint placement as a variable in the ILP:

New WGGO variables:

```
ckpt_tier1[layer] ∈ {0, 1}    # inject GPU→CPU copy after this layer's backward?  
ckpt_tier2[step_interval] ∈ Z  # Tier 2 interval (CPU→NVMe)
```

Objective addition:

```
minimize: step_time + λ × checkpoint_overhead
```

Constraints:

```
# At least one Tier 1 checkpoint per step (for recovery)
```

```
Σ ckpt_tier1[layer] ≥ 1
```

```
# Checkpoint must not conflict with CPDT communication
```

```
∀ l: ckpt_tier1[l] = 1 → no reducescatter in flight for layer l
```

```
# Total Tier 1 data ≤ PCIe bandwidth budget
```

```
Σ ckpt_tier1[l] × layer_param_size[l] / pcie_bandwidth ≤ max_ckpt_time
```

The compiler generates checkpoint instructions as part of the backward IR. The

`cudaMemcpyAsync` calls are interleaved with compute at the IR level — the same technique CPDT uses for communication overlap. The cost model determines which layers' parameters to stream to CPU during which compute phase, ensuring the PCIe copies overlap with GPU matmuls.

WGGO selects the optimal checkpoint layers: Not all layers need checkpointing every step. The ILP selects a subset of layers that captures ~95% of model state diversity (using weight importance from CEP) while fitting within the PCIe bandwidth budget. Full checkpoints occur at Tier 2 intervals.

2.2 Elastic Topology Recompile

The problem: When a node fails, the CPDT topology is invalidated. The reduce-scatter expects N GPUs; it gets N-k. A fully static binary cannot adapt.

CPFT's approach: The compiler generates **multiple topology variants** at compile time and a lightweight runtime selector:

Compile-time:

WGGO solves for the primary topology: 1024 GPUs

WGGO also pre-solves for degraded topologies:

- 1024 → 1016 (1 node down, DP shrinks by 8)
- 1024 → 1008 (2 nodes down)
- 1024 → 992 (4 nodes down)

Each topology variant generates:

- A different CPDT communication schedule
- Adjusted grad_accum_steps (preserve effective batch size)
- Adjusted FASE accumulation count

All variants are compiled into the SAME binary as separate function pointers.

Runtime:

When heartbeat detects failure:

1. ElasticWorld computes new_dp_replicas
2. Lookup pre-compiled variant for closest matching topology
3. Hot-swap function pointer table (zero recompilation)
4. Load checkpoint from Tier 1 (CPU DRAM – microseconds)
5. Resume training at next step

If topology falls below any pre-compiled variant:

6. Trigger background micro-recompilation (WGGO Level 1 DP only – ~50ms)
7. Generate new communication schedule for exact remaining GPUs
8. Hot-swap when ready (training continues on degraded variant until then)

Why this works for NSL: The compiler already knows the full model topology and all CPDT parameters. Pre-solving for 3-4 degraded topologies adds ~5 seconds to compilation time (WGGO is fast). The binary includes all variants — no JIT compiler needed at runtime.

The micro-recompilation path: For exotic failure patterns (e.g., half of a TP group fails, which invalidates tensor parallelism), CPFT triggers a partial WGGO re-solve at runtime. Because WGGO Level 1 (inter-layer DP) takes ~50ms, this recompilation is faster than loading a checkpoint from NVMe. The runtime only recompiles the communication schedule, not the compute kernels — those are topology-independent.

2.3 Redundant State Placement via Compilation Analysis

The problem: Gemini (SOSP '23) places checkpoint replicas across machines to maximize recovery probability. But the placement depends on the parallelism topology, which the placement algorithm discovers at runtime.

CPFT's approach: The compiler knows the parallelism topology at compile time and generates an optimal replica placement plan:

Given CPDT topology: TP=8, PP=1, DP=128 on 1024 GPUs

Compiler analysis:

- TP group members must NOT checkpoint to each other (correlated failure)
- ZeRO-3 optimizer shards are unique per DP rank – must be replicated
- Model weights are replicated within DP group – only one copy needed

Compiled placement plan:

For each DP rank r:

Tier 1 checkpoint → CPU DRAM of local node (always)

Tier 1 replica → CPU DRAM of node in different TP group within same rack

For each DP group:

Tier 2 checkpoint → NVMe of first rank in group

Tier 2 replica → NVMe of first rank in DIFFERENT DP group (cross-rack)

Tier 3: standard remote storage (S3/shared FS)

The placement plan is a compile-time constant embedded in the checkpoint manager. No runtime discovery of topology is needed.

3. NSL Language Surface

```
ns!

@cpdt(cluster = { gpus = 1024, intra_bw = "900GB/s", inter_bw = "100GB/s" })
@cpft(
  checkpoint:
    tier1_interval = 1          # every step
    tier2_interval = 100       # every 100 steps
    tier3_interval = 10000     # every 10K steps
    async = true              # non-blocking GPU→CPU
    optimal_layers = auto      # WGGO selects which layers to checkpoint per step
  elastic:
    degraded_topologies = [1016, 1008, 992] # pre-compile these variants
    min_dp_replicas = 64       # stop training below this threshold
    recompile_threshold = 32    # trigger micro-recompilation if >32 GPUs lost
)
train(model=m, epochs=1):
  ...
```

4. Implementation Plan

Component	Location	LOC (est.)
WGGO checkpoint injection	<code>ns1-codegen/src/cpft_checkpoint.rs</code>	500
Multi-topology compilation	<code>ns1-codegen/src/cpft_elastic.rs</code>	600
Replica placement planner	<code>ns1-codegen/src/cpft_placement.rs</code>	300
Runtime topology switcher	<code>ns1-runtime/src/elastic/switcher.rs</code>	400
Micro-recompilation engine	<code>ns1-codegen/src/cpft_recompile.rs</code>	500
Total		~2,300 LOC

Paper 12: Compile-Time Quantization Synthesis (CTQS)

How the Compiler Fuses Sub-Byte Weight Packing

1. The Problem

NSL already has quantization as a first-class language feature (Chapter 6: `quant` keyword, schemes `int4/int8/fp8/nf4/gptq/awq`, modes `static/dynamic/aware/weight_only`) and FP8/FP4 codegen (M35 with PerTensor/PerBlock/PerChannel scaling, Hadamard transform). But the current implementation delegates to runtime dequantization — the quantized weights are stored packed, then dequantized to BF16 before matmul.

The industry relies on opaque Python scripts (AutoGPTQ, AutoAWQ, bitsandbytes) that run outside the compilation pipeline. The dequantization overhead can consume 15-30% of inference time for INT4 models.

2. Three Compiler-Native Innovations

2.1 Weight-Spectral Precision Selection

The problem: AWQ uses activation-aware weighting; GPTQ uses second-order Hessian approximation. Both require calibration data and runtime Python. NSL can do this at compile time.

CTQS's approach: When `--weights` is provided, the compiler performs per-layer spectral analysis to select quantization precision:

```

Per-layer precision selection (compile-time):
For each weight matrix W:
    singular_values = svd(W).S
    effective_rank = count(S > 0.01 × S[0])
    spectral_condition = S[0] / S[effective_rank]

    if spectral_condition < 10:      # well-conditioned, tolerant
        precision = INT4, group_size = 128
    elif spectral_condition < 100: # moderate
        precision = INT8, per-channel scaling
    else:                          # ill-conditioned, sensitive
        precision = FP8 E4M3, per-tensor scaling

```

For attention projections (W_q, W_k, W_v, W_o): typically INT4 with group=128 (well-conditioned).

For FFN gate/up/down: often INT4 or INT8 (depends on activation structure).

For embeddings: FP8 (high spectral condition, outlier-sensitive).

For LM head: FP8 or BF16 (maximum precision for output quality).

These are compile-time constants. The compiled binary contains per-layer precision as kernel configuration parameters.

2.2 Fused Dequant-MMA Kernel Synthesis

The problem: Standard INT4 inference loads packed INT4 weights, unpacks them to FP16/BF16 in a separate kernel, then runs the matmul. Two kernel launches, one HBM round-trip for the unpacked weights.

CTQS's approach: The compiler generates a single kernel that loads packed INT4 weights and performs dequantization **in registers** before the tensor core MMA instruction:

```

Fused Dequant-MMA Kernel (compiler-generated PTX):
// Load INT4 weights (packed 2 per byte)
ld.global.b32 r0, [weight_ptr]           // 8 INT4 values in one 32-bit register

// Extract INT4 values
prmt.b32 r1, r0, 0, 0x5410               // extract lower 4 values
prmt.b32 r2, r0, 0, 0x7632               // extract upper 4 values

// Dequantize: scale × (int4_val - zero_point)
// Scale and zero_point are compile-time constants per group
cvt.f16.s4 r3, r1                        // INT4 → FP16 (hardware instruction on
Blackwell)
fma.f16 r4, r3, scale, neg_zero_offset  // scale × val + bias

// Now r4 contains dequantized FP16 values in registers
// Feed directly to tensor core MMA
mma.sync.aligned.m16n8k16.f32.f16.f16.f32 ...

```

On Blackwell GPUs: The B200 has native INT4×FP8 MMA instructions. CTQS generates PTX that uses these directly, bypassing dequantization entirely:

```

// Blackwell INT4 MMA (no dequantization needed!)
mma.sync.aligned.m16n8k64.f32.e4m3.e4m3.f32 d, a_int4, b_fp8, c

```

On Hopper GPUs (no native INT4 MMA): CTQS generates the dequant-in-register approach above, fusing it with the FP16 MMA.

The compiler selects the kernel variant based on the target GPU — `--target h100` generates Hopper-optimized dequant-MMA, `--target b200` generates Blackwell native INT4 MMA. No runtime dispatch.

2.3 Quantization-Aware Training as a Compiler Pass

The problem: QAT in PyTorch requires inserting fake-quantize nodes throughout the model, running calibration, and managing straight-through estimator (STE) gradients. This is manual and error-prone.

CTQS's approach: When the `quant` block specifies `mode = aware`, the compiler inserts quantization-dequantization nodes into the Wengert graph automatically:

```

nsl

quant(scheme=int4, mode=aware, calibration=entropy):
    model = m
    calibration_data = val_data
    target = rtx5070ti

```

The compiler:

1. Inserts fake-quantize nodes after every weight read in the forward Wengert graph
2. Source AD generates backward passes with STE (straight-through estimator) automatically
3. The cost model determines which layers benefit from QAT vs PTQ
4. WGGO integrates quantization precision as variables in the Level 2 ILP

Fused QAT kernels: The fake-quantize (quant→dequant round-trip) is fused into the matmul prologue. No separate kernel launch, no intermediate tensor. The forward matmul loads weights, applies fake-quant in registers, and proceeds to MMA — identical to the inference dequant-MMA kernel but with gradient passthrough.

3. NSL Language Surface

```
ns!

# Post-training quantization with weight-spectral precision selection
quant(scheme=auto, mode=weight_only):
    model = m
    calibration_data = val_data
    precision = spectral           # compiler selects per-layer from weight analysis
    target = h100                 # generate Hopper-optimized dequant-MMA kernels

# Quantization-aware training
quant(scheme=int4, mode=aware, group_size=128):
    model = m
    calibration = entropy
    # Source AD generates STE backward automatically
    # Fake-quant fused into matmul prologue
```

```
bash

$ nsl build --quantize=spectral --weights model.safetensors --target h100 model.nsl

=== CTQS Quantization Report ===
Weight-spectral analysis:
  embed:          FP8 E4M3 (spectral condition: 312)
  blocks.0-31 Wq: INT4 group=128 (condition: 8.3)
  blocks.0-31 Wk: INT4 group=128 (condition: 7.1)
  blocks.0-31 FFN: INT4 group=64 (condition: 12.4)
  lm_head:        BF16 (preserved for output quality)

Model size: 14 GB (BF16) → 4.2 GB (mixed INT4/FP8/BF16)
Compression ratio: 3.3×
Kernel type: Fused dequant-MMA (Hopper H100, FP16 MMA with register dequant)
Estimated quality loss: <0.5% perplexity degradation (spectral-guided)
```

4. Implementation Plan

Component	Location	LOC (est.)
Weight-spectral selector	nsl-codegen/src/ctqs_spectral.rs	400
Fused dequant-MMA PTX gen	nsl-codegen/src/ctqs_dequant_mma.rs	700
INT4/NF4 packing codegen	nsl-codegen/src/ctqs_packing.rs	400
QAT fake-quant insertion	nsl-codegen/src/ctqs_qat.rs	500
STE backward generation	nsl-codegen/src/ctqs_ste.rs	300
WGGO integration	nsl-codegen/src/wggo.rs	200
Total		~2,500 LOC

Paper 13: Compiler-Native Architecture Generalization (CNAG)

Beyond Transformers: SSMs, Multimodal, and Hybrid Architectures

1. The Problem

Our previous papers (CSHA, PCA, CFIE) are transformer-specific — they fuse attention boundaries, pack sequences for attention masking, and generate persistent attention+FFN decode blocks. But the frontier is shifting:

- **Mamba/SSMs:** Replace attention with selective state space models using parallel associative scans
- **Hybrid architectures:** Jamba (Mamba+Attention), Gemma 4 (alternating local/global attention)
- **Multimodal:** Gemma 4, LLaMA 4 — process images, audio, video alongside text

NSL already has multimodal primitives (M48: PatchEmbedConfig, MelConfig) and could theoretically express SSMs via its model/function syntax. But there's no compiler-native optimization for associative scans (the SSM bottleneck, equivalent to FlashAttention for transformers) or heterogeneous data streams.

2. Three Compiler-Native Innovations

2.1 Compiled Associative Scan Fusion

The problem: The selective SSM recurrence $h_t = A_t \times h_{t-1} + B_t \times x_t$ is trained via parallel associative scan. PyTorch only supports this via custom CUDA kernels (mamba-ssm) or `torch._higher_order_ops.associative_scan` (experimental, requires torch ≥ 2.9). The scan must be fused with input projections, discretization (Δ), and output projection — Mamba's original paper calls this "kernel fusion" as one of three critical optimizations.

CNAG's approach: The compiler recognizes the associative scan pattern in the Wengert graph and generates a fused kernel:

nsi

```
# User writes the SSM layer naturally
model MambaBlock(d_model: int, d_state: int, d_conv: int):
    A: Tensor = randn([d_model, d_state])
    D: Tensor = ones([d_model])
    conv1d: Conv1D = Conv1D(d_model, d_conv)
    x_proj: Linear = Linear(d_model, d_state * 2) # B and C projections
    dt_proj: Linear = Linear(d_model, d_model)    # Δ projection

    fn forward(x: Tensor<[batch, seq, d_model]>) -> Tensor<[batch, seq, d_model]>:
        let x_conv = self.conv1d(x)
        let x_act = silu(x_conv)

        let delta = softplus(self.dt_proj(x_act)) # Δ
        let (B, C) = self.x_proj(x_act).chunk(2, dim=-1) # B, C

        let A_bar = exp(einsum("bd,dn->bdn", delta, self.A)) # discretized A
        let B_bar = einsum("bd,bd,bn->bdn", delta, x_act, B) # discretized B × C

        # The compiler recognizes this as an associative scan pattern:
        let h = associative_scan(A_bar, B_bar, dim=1) # parallel prefix
        let y = einsum("bdn,bn->bd", h, C) # output projection

        return y + x_act * self.D # skip connection
```

The compiler detects the associative scan and generates a fused kernel:

```
Fused Mamba Kernel (compiler-generated):
1. Load x tile into SMEM
2. Conv1D in SMEM (causal, d_conv=4)
3. SiLU activation in registers
4. x_proj matmul: compute B, C in SMEM
5. dt_proj matmul: compute Δ in SMEM
6. Discretize: A_bar = exp(Δ × A), B_bar = Δ × x × B – in registers
7. Parallel associative scan over sequence dimension:
   - Blelloch-style up-sweep/down-sweep in SMEM
   - Tiled: each CTA processes TILE tokens, inter-CTA reduction via global
memory
8. Output projection: y = h × C – in SMEM
9. Skip connection + store to HBM

Total HBM traffic: read x, read weights, write y
Eliminated: ALL intermediate tensors (conv output, projections, Δ, A_bar, B_bar,
h)
```

This is the SSM equivalent of CSHA — fusing across the entire Mamba block's computational boundaries. Mamba-3's team built these kernels by hand using a mix of Triton, TileLang, and CuTe DSL. CNAG generates them automatically from the NSL source.

Source AD for SSM backward: The associative scan has a known adjoint (reverse scan). Source AD (M40) generates the backward pass automatically, including the fused $dA/dB/dC/d\Delta$ gradients through the scan.

2.2 Heterogeneous Modality Pipeline

The problem: Multimodal models process different input types (text tokens, image patches, audio frames) with different embedding dimensions, sequence lengths, and compute characteristics. A 224×224 image produces 196 patches (14×14); a 30-second audio clip produces ~3000 mel frames. These heterogeneous "tokens" are concatenated and processed by the transformer, but their different sizes create load-imbalance problems.

CNAG's approach: The compiler analyzes the multimodal pipeline at compile time and generates modality-specific kernels:

```
ns!

model MultimodalLM(d_model: int, n_layers: int):
    text_embed: Embedding = Embedding(vocab_size, d_model)
    vision_encoder: PatchEmbed = PatchEmbed(image_size=224, patch_size=16, embed_dim=d_model)
    audio_encoder: MelEmbed = MelEmbed(n_mels=80, embed_dim=d_model)

    blocks: [TransformerBlock; n_layers] = ...

    fn forward(text: Tensor, images: Tensor?, audio: Tensor?) -> Tensor:
        let text_tokens = self.text_embed(text)           # [B, S_text, D]
        let image_tokens = self.vision_encoder(images)     # [B, S_image, D]
        let audio_tokens = self.audio_encoder(audio)       # [B, S_audio, D]

        let combined = concat([text_tokens, image_tokens, audio_tokens], dim=1)
        let position_ids = multimodal_position_ids(text, images, audio)

        for block in self.blocks:
            combined = block.forward(combined, position_ids)

        return combined
```

The compiler generates:

1. **Modality-aware PCA packing:** Different modalities get different segment IDs. The PCA attention kernel ensures text tokens don't attend to image patches unless the model's attention pattern allows it. The segment-ID vector now encodes modality type + document boundary.
2. **Modality-specific position embeddings:** RoPE frequencies differ for text (token-level) vs image (2D patch positions) vs audio (temporal frames). The fused RoPE kernel applies modality-specific rotations based on the segment-ID modality field — a compile-time branch in the kernel.
3. **Heterogeneous batch scheduling:** The cost model knows that vision encoder patches are compute-bound (conv2d) while text tokens are memory-bound (embedding lookup). The compiled pipeline schedules vision encoding on one CUDA stream while text embedding runs on another, overlapping compute and memory operations.

2.3 Hybrid Architecture Support (Mamba + Attention)

The problem: Jamba and Gemma 4 alternate between attention layers and SSM/sliding-window layers. The compiler needs to handle both in a single model.

CNAG's approach: WGGO's per-layer ILP naturally handles heterogeneous layer types. Each layer's decision variables include the layer type:

```
WGGO Level 2 for hybrid models:
```

```
For each layer l:
```

```
    type[l] ∈ {attention, ssm, sliding_window} # determined by model definition
```

```
    if type[l] == attention:
```

```
        # Standard CSHA variables: fusion level, head pruning, etc.
```

```
        c[l] ∈ {0, 1, 2, 3}
```

```
        h[l,k] ∈ {0, 1}
```

```
    elif type[l] == ssm:
```

```
        # SSM-specific variables: scan tile size, conv fusion, state precision
```

```
        scan_tile[l] ∈ {64, 128, 256, 512}
```

```
        conv_fused[l] ∈ {0, 1}
```

```
        state_precision[l] ∈ {fp16, fp32}
```

```
    elif type[l] == sliding_window:
```

```
        # Window attention variables
```

```
        window_size[l] = 4096 # fixed by architecture
```

```
        # Use local attention kernel variant (no global KV cache)
```

The cost model evaluates SSM layers using a different roofline profile: SSMs are memory-bound during the scan phase (sequential data dependency) and compute-bound during projections. The ILP balances these differently than attention layers.

Inference for hybrid models: CFIE generates different persistent decode kernels for different layer types:

- **Attention layers:** CSHA-L2 fused decode (as in Paper 7)
- **SSM layers:** Recurrent mode — $O(1)$ state update per token (no KV cache, constant memory)
- **Sliding window layers:** Local attention over fixed window (no growing KV cache for these layers)

The compiled inference binary alternates between these kernel types based on the model's layer schedule — a compile-time constant.

3. NSL Language Surface

ns!

```
# Hybrid Mamba-Attention model
model JambaBlock(d_model: int, n_heads: int, d_state: int, layer_type: str):
    if layer_type == "attention":
        attn: GroupedQueryAttention = GroupedQueryAttention(d_model, n_heads, n_head
    elif layer_type == "ssm":
        ssm: MambaBlock = MambaBlock(d_model, d_state, d_conv=4)

    ffn: SwiGLUFFN = SwiGLUFFN(d_model, d_model * 4)
    norm1: RMSNorm = RMSNorm(d_model)
    norm2: RMSNorm = RMSNorm(d_model)

    fn forward(x: Tensor, kv_cache: KVCache?) -> Tensor:
        let h = self.norm1(x)
        if self.layer_type == "attention":
            h = self.attn(h, kv_cache)
        else:
            h = self.ssm(h)
        let x = x + h
        let h = self.norm2(x)
        return x + self.ffn(h)

model Jamba(d_model: int, n_layers: int):
    # Alternating: attention, ssm, ssm, attention, ssm, ssm, ...
    layer_types = ["attention" if i % 3 == 0 else "ssm" for i in 0..n_layers]
    blocks = [JambaBlock(d_model, 32, 16, lt) for lt in layer_types]
    ...
```

4. Implementation Plan

Component	Location	LOC (est.)
Associative scan detector	ns1-codegen/src/cnag_scan_detect.rs	400
Fused scan kernel gen	ns1-codegen/src/cnag_scan_kernel.rs	700
Scan backward (source AD)	ns1-codegen/src/cnag_scan_adjoint.rs	400
Multimodal pipeline	ns1-codegen/src/cnag_multimodal.rs	500
Modality-aware PCA	ns1-codegen/src/cnag_modality_pca.rs	300
Hybrid layer WGGO	ns1-codegen/src/wggo.rs	300
SSM inference (recurrent)	ns1-codegen/src/cnag_ssm_inference.rs	400
Total		~3,000 LOC

Conclusion: The Complete Architecture

With these final three papers, NSL covers every edge case in the modern AI lifecycle:

#	Paper	Domain	LOC
1	WRGA	PEFT / Adapters	~2,500
2	CSHA	Attention Fusion	~2,800
3	CFTP (FASE+PCA)	Gradient Accumulation + Packing	~2,250
4	CEP	Pruning + NAS	~2,850
5	CPDT	Distributed Training	~3,100
6	WGGO	Global ILP Solver	~3,000
7	CFIE	Inference Optimization	~3,100
8	CPKD	Knowledge Distillation	~2,350
9	CIDP	Data Quality + Filtering	~2,100
10	CFA	RLHF / DPO / GRPO Alignment	~3,150
11	CPFT	Fault Tolerance	~2,300
12	CTQS	Quantization Synthesis	~2,500
13	CNAG	SSMs + Multimodal	~3,000
TOTAL			~35,000 LOC

The thirteen papers, totaling ~35,000 lines of estimated Rust code across `ns1-codegen` and `ns1-runtime`, address every category:

DATA:

CIDP (quality, filtering, curriculum, mixing)

ARCHITECTURE:

CEP (pruning, NAS), CNAG (SSMs, multimodal, hybrid)

TRAINING:

WRGA (PEFT), CSHA (attention), FASE (accumulation), PCA (packing),
CPDT (distributed), CPKD (distillation), CFA (alignment)

INFERENCE:

CFIE (serving, speculation, fused sampling)

DEPLOYMENT:

CTQS (quantization, weight packing, dequant-MMA)

RESILIENCE:

CPFT (checkpointing, elastic topology, fault recovery)

SOLVER:

WGGO (global ILP that composes all 12 techniques)

One thesis, thirteen papers: Every decision that the modern AI stack makes at runtime — from data quality scoring to attention fusion to gradient accumulation to speculative decoding to fault recovery — NSL makes at compile time. The compiled binary IS the globally-optimal training, alignment, and inference plan, resilient to hardware failures, general across architectures, and deployable at sub-byte precision.

References

Paper 11 (CPFT)

1. Wang, Z., et al. "Gemini: Fast Failure Recovery in Distributed Training with In-Memory Checkpointing." SOSP 2023.
2. ByteDance. "ByteRobust: Robust LLM Training Infrastructure." 2025.
3. Lian, X., et al. "Universal Checkpointing: Efficient and Flexible Checkpointing for Large Scale Distributed Training." USENIX ATC 2025.
4. Meta. "Revisiting Reliability in Large-Scale Machine Learning Research Clusters." 2024.

Paper 12 (CTQS)

5. Frantar, E., et al. "GPTQ: Accurate Post-Training Quantization for Generative Pre-Trained Transformers." ICLR 2023.
6. Lin, J., et al. "AWQ: Activation-Aware Weight Quantization for LLM Compression." MLSys 2024.
7. Dettmers, T., et al. "QLoRA: Efficient Finetuning of Quantized LLMs." NeurIPS 2023.
8. NVIDIA. "Blackwell Architecture Whitepaper: NVFP4 and INT4 MMA." 2024.

Paper 13 (CNAG)

9. Gu, A., Dao, T. "Mamba: Linear-Time Sequence Modeling with Selective State Spaces." ICLR 2024.
10. Dao, T., Gu, A. "Transformers are SSMs: Generalized Models Through Structured State Space Duality." ICML 2024.
11. Gu, A., et al. "Mamba-3." Together AI, 2026.
12. Lieber, O., et al. "Jamba: A Hybrid Transformer-Mamba Language Model." 2024.